

Guia Oficial de Engenharia Back-end — Santo Pegasus Soluciones

Versão do Documento: 3.0.0 (Edição Global de Engenharia)

Última Atualização: Outubro de 2025

Departamento: Engenharia de Software / Centro de Excelência Técnica

Idioma: Português Brasileiro (PT-BR)

1. Introdução Profunda: A Gênese da Excelência

1.1. Nossa História e Missão Tecnológica

A Santo Pegasus Soluciones nasceu da necessidade de convergir dois setores críticos da economia brasileira: o ecossistema financeiro (FinTech) e a infraestrutura de saúde (HealthTech). Desde a nossa fundação, entendemos que o software não é apenas um suporte, mas o núcleo de confiança entre a instituição e o cidadão.

Nossa missão tecnológica é "Construir sistemas resilientes que protejam a integridade dos dados e garantam a disponibilidade absoluta de serviços vitais". Para a Santo Pegasus, um milissegundo de latência em um processamento de Pix ou uma falha de sincronização em um prontuário médico não são apenas erros técnicos; são impactos diretos na vida e no patrimônio de nossos usuários.

1.2. Cultura "Engineering Excellence"

A cultura de excelência em engenharia (Engineering Excellence) na Santo Pegasus baseia-se em quatro pilares inegociáveis:

- Propriedade Técnica (Ownership):** O desenvolvedor é responsável pelo ciclo de vida completo de seu código, do design ao monitoramento em produção.
- Simplicidade Radical:** Resolver problemas complexos com o mínimo de código necessário. Código que não existe não possui bugs.
- Segurança por Design:** A conformidade com a LGPD e a proteção contra ameaças cibernéticas começam na primeira linha de código, não no deploy.
- Evolução Contínua:** Adotamos o "Kaizen" — melhoria constante. O Guia de Engenharia é um organismo vivo que reflete o estado da arte da tecnologia.

2. Princípios de Engenharia e Design de Software (SOLID)

Na Santo Pegasus, os princípios SOLID não são sugestões; são o fundamento de nossa arquitetura. Abaixo, detalhamos cada princípio com implementações modernas em Java 17/21.

2.1. S — Single Responsibility Principle (SRP)

Definição: Uma classe deve ter uma, e apenas uma, razão para mudar.

Cenário de Refatoração:

Legado (Incorreto): Um `PaymentService` que valida o usuário, calcula taxas, salva no banco e envia e-mail.

Clean Code (Correto):

```
```java
// Responsabilidade Única: Apenas orquestra o fluxo de pagamento
@Service
@RequiredArgsConstructor
public class PaymentService {
 private final PaymentValidator validator;
 private final FeeCalculator feeCalculator;
 private final PaymentRepository repository;
 private final NotificationService notificationService;
 @Transactional
 public PaymentResponse process(PaymentRequest request) {
 validator.validate(request);
 var fee = feeCalculator.calculate(request.amount());
 var entity = repository.save(new PaymentEntity(request, fee));
 notificationService.sendReceipt(entity);
 return PaymentResponse.from(entity);
 }
}
```

## 2.2. O — Open/Closed Principle (OCP)

**Definição:** Objetos devem estar abertos para extensão, mas fechados para modificação.

**Aplicação com Java 17 Records e Sealed Interfaces:**

```
```java
public sealed interface PaymentMethod permits Pix, Boleto, CreditCard {}
public record Pix(String key) implements PaymentMethod {}
public record Boleto(String barcode) implements PaymentMethod {}
@Service
public class PaymentProcessor {
    private final Map<Class<? extends PaymentMethod>, PaymentStrategy> strategies;
    public void process(PaymentMethod method) {
```

```

        strategies.get(method.getClass()).execute(method);
    }
}
...

```

Isso permite adicionar novos métodos de pagamento sem tocar na classe `PaymentProcessor`.

2.3. L — Liskov Substitution Principle (LSP)

Definição: Subclasses devem ser substituíveis por suas classes base sem alterar a correção do programa.

Na Santo Pegasus, garantimos que nossas implementações de repositório (SQL vs NoSQL) sigam a mesma interface de contrato, assegurando que o domínio não sofra com a troca de tecnologia de persistência.

2.4. I — Interface Segregation Principle (ISP)

Definição: Muitas interfaces específicas são melhores do que uma interface geral.

Evitamos interfaces "Gordas". Em vez de `IUserService`, temos `IUserAuthenticator`, `IUserProfileManager` e `IUserDeletor`.

2.5. D — Dependency Inversion Principle (DIP)

Definição: Dependenda de abstrações, não de implementações concretas.

Utilizamos a Injeção de Dependência do Spring para garantir que nossas classes de `Service` dependam apenas de `Interfaces` de repositórios ou clientes externos.

3. Ecossistema Spring Boot 3.x e Java Moderno

3.1. Java 17/21: O Novo Padrão

- **Records:** Substituem classes POJO/Lombok para DTOs. São imutáveis por padrão e reduzem o boilerplate.
- **Sealed Classes:** Utilizadas para representar hierarquias de domínio fechadas (ex: Status de Pedido).
- **Pattern Matching for Switch:** Melhora a legibilidade em fluxos de decisão complexos.
- **Virtual Threads (Java 21):** Para serviços de alto throughput, permitindo escalar a execução concorrente sem sobrecarregar o SO.

3.2. Spring Boot Actuator e Observabilidade

Todo microserviço Santo Pegasus deve expor os endpoints de saúde e métricas:

- `/actuator/health`: Monitoramento de prontidão (Liveness/Readiness).
- `/actuator/prometheus`: Exportação de métricas para o Grafana.

- **Custom Metrics:** Utilizamos `MeterRegistry` do Micrometer para medir o volume financeiro transacionado em tempo real (R\$).

4. Arquitetura em Camadas e Hexagonal

Nossa arquitetura visa o desacoplamento total do Framework.

4.1. As Camadas

1. **Domínio (Domain):** Onde reside a inteligência do negócio. Classes PURE JAVA. Sem anotações do Spring ou JPA. Aqui definimos as `Entities` e as `Domain Rules`.
2. **Aplicação (Application):** Contém os `Use Cases`. Orquestra a lógica de negócio chamando as `Ports` (Interfaces).
3. **Infraestrutura (Infrastructure):** Camada de detalhes. Implementações de `Adapters` (Repositories, WebClients, Mensageria).

4.2. Regra de Dependência

As dependências devem sempre apontar para o Domínio. O Domínio nunca conhece o Banco de Dados ou a API REST. Isso é garantido através do padrão Ports and Adapters.

5. Estratégia RAG (IA) Avançada

Para o processamento de dados não estruturados na Santo Pegasus, implementamos pipelines de RAG (Retrieval-Augmented Generation).

5.1. Pipeline de Dados

1. **Semantic Chunking:** Em vez de divisões fixas, utilizamos análise de parágrafos para manter o contexto semântico.
2. **Embedding Model:** Padronizamos o `text-embedding-004` via AWS Bedrock ou OpenAI.
3. **Vector Store:** Uso de Qdrant em clusters de alta disponibilidade.
4. **Hybrid Search:** Combinamos busca vetorial (similaridade) com busca de texto completo (BM25) para aumentar a precisão em termos técnicos de saúde.

5.2. Prompt Engineering e Segurança

- **System Prompts:** Devem incluir restrições estritas de "Hallucination Check".
- **PII Filtering:** Antes de enviar qualquer dado para o LLM, o conteúdo passa por uma camada de anonimização para proteger dados de pacientes sob a LGPD.

6. Segurança e LGPD de Ponta a Ponta

6.1. Autenticação e Autorização (OAuth2/JWT)

- Utilizamos Keycloak ou AWS Cognito como Identity Provider (IdP).
- JWT Claims: Devem conter `scopes` específicos e `roles`. Nunca incluir senhas ou dados sensíveis no Payload do JWT.

6.2. Proteção de Dados (PII) e Criptografia

- Em Trânsito: TLS 1.3 obrigatório em todas as comunicações.
- Em Repouso: Criptografia de colunas sensíveis (CPF, Valor de Saldo) no PostgreSQL utilizando `pgcrypto` ou chaves gerenciadas no AWS KMS.
- Anonimização: Em ambientes de Homologação, os dados de produção devem ser obrigatoriamente "mascarados" ou "anonimizados".

7. Comunicação e Resiliência

7.1. Protocolos

- REST/JSON: Para integrações externas e Front-end.
- gRPC: Para comunicação inter-serviços de alta performance, utilizando Protobuf para reduzir o overhead de rede.

7.2. Padrões de Estabilidade (Resilience4j)

- Circuit Breaker: Se um serviço externo falhar, o circuito abre para evitar o efeito cascata.
- Rate Limiting: Proteção contra ataques de negação de serviço e controle de custos de API.
- Retry: Implementado com `Exponential Backoff` para falhas transitórias.

8. Banco de Dados e Migrations

8.1. Evolução de Esquema com Flyway

Toda alteração em banco de dados deve ser versionada através de scripts SQL no diretório `src/main/resources/db/migration`.

- Naming Convention: `V{VERSAO}\_\_{DESCRICAO}.sql`.

8.2. Performance JPA/Hibernate

- N+1 Problem: Uso obrigatório de `EntityGraph` ou `JOIN FETCH` em queries complexas.
- ReadOnly Transactions: Métodos de consulta devem ser anotados com `@Transactional(readOnly = true)` para otimizar o uso do Pool de conexões.

9. Estratégia de Testes e Qualidade

9.1. A Pirâmide de Testes Santo Pegasus

- 1. Testes Unitários (70%):** Testam a lógica pura dos Services e Entities. Mockamos todas as dependências com Mockito.
- 2. Testes de Integração (20%):** Validam a integração com o Banco de Dados e APIs externas. Utilizamos Testcontainers para subir instâncias reais de PostgreSQL ou Redis durante os testes.
- 3. Testes E2E/Contrato (10%):** Validam o fluxo completo. Utilizamos Pact.io para garantir que mudanças no Back-end não quebrem o Front-end ou outros serviços.

10. Infraestrutura e CI/CD

10.1. Dockerização Otimizada

Nossos Dockerfiles utilizam Multi-stage builds para garantir imagens leves e seguras:

```dockerfile

#### Stage 1: Build

FROM maven:3.9-eclipse-temurin-21 AS build

COPY . .

RUN mvn clean package -DskipTests

#### Stage 2: Runtime

FROM eclipse-temurin:21-jre-jammy

COPY --from=build /target/.jar app.jar

ENTRYPOINT ["java", "-jar", "/app.jar"]

```

10.2. Deploy e Monitoramento

- **Plataforma:** AWS ECS (Elastic Container Service) com Fargate.
- **Deployment:** Preferência por Blue-Green para serviços críticos. O tráfego só é virado após o "Green" passar em todos os Health Checks.
- **Alerting:** Alertas configurados no Prometheus para latência P99 > 500ms e taxa de erro 5xx > 1%.

11. Apêndices e Glossário

11.1. Ferramentas Homologadas

- IDE: IntelliJ IDEA (Recomendado) ou VS Code.
- Gerenciamento de Dependências: Maven 3.8+.
- Banco de Dados: PostgreSQL 15+, Redis 7+.
- Mensageria: RabbitMQ ou AWS SQS.

11.2. Dicionário Técnico (Santo Pegasus)

- Agendio: Nosso sistema core de agendamentos de saúde.
- PegasusPay: Nosso gateway interno de pagamentos multicanal.
- IdP (Identity Provider): Provedor de identidade centralizado.
- Idempotência: Garantia de que uma operação pode ser repetida sem efeitos colaterais indesejados (vital para pagamentos em R\$).

Este documento é de propriedade da Santo Pegasus Soluciones. A divulgação externa sem autorização prévia é proibida.